

JTBD FEEDBACK LOOP

Interview Preparation Guide

Invoca — Applied AI Analyst / JTBD Feedback Loop Architect

Erwin M. McDonald · Fort Worth, TX · March 2026

This document gives you everything you need to speak with authority about your 7-step JTBD pipeline — technically and non-technically. It covers the Anthropic API and SDK precisely, articulates the specific gap your framework identifies in Invoca's product, and is backed by Invoca's public product documentation and your own POC artifacts.

SECTION 1: What You Built — Lead With This

Start every answer here, regardless of who is in the room. This is your 30-second frame.

"I built a 7-step AI pipeline that reads a customer success call transcript and automatically extracts every signal that matters — bugs, churn risks, competitor mentions, pricing friction, feature requests, and wins. Each signal is scored for confidence, routed to the right stakeholder team with a deadline, and the loop closes when the CSM receives a confirmation receipt. Nothing falls through the cracks."

Core technical stack:

- Language: Python 3.11+ · AI: Anthropic claude-sonnet-4-6 via Messages API (POST /v1/messages)
- SDK: anthropic Python package — `client.messages.create()`
- Schema: Typed Python dataclasses — `ExtractedInsight`, `RoutedAlert` — versioned alongside the prompt
- Frontend POC: React + Recharts — live Anthropic API calls from the browser client
- Output: JSON alerts — webhook-ready for Salesforce task creation, Slack, or Jira ticket

SECTION 2: The 7-Step Pipeline — Both Audiences

For each step: technical explanation, plain-English executive version, the API/SDK layer, and the Invoca gap where applicable.

STEP 1 — INGEST

TECHNICAL

Raw call transcript — a plain text string — is passed as input. In the POC: `transcript = open("call.txt").read()`. In production it comes from Invoca's Call Ingestion API or a call recording platform. No preprocessing, no tokenization at this layer. The string becomes the user message content in the Anthropic SDK call. Production adds a PII redaction layer before the text touches the model.

NON-TECHNICAL / EXECUTIVE

The call ends and the system immediately picks it up — like a smart mailbox. The moment the transcript arrives, the system takes over. Nothing waits for a CSM to sit down and type notes. Zero manual intake required.

API / SDK: *Python file I/O: `transcript = open("call.txt").read()` — or from Invoca stream: `invoca_api.get_transcript(call_id)`. Passed as `messages=[{"role": "user", "content": transcript}]` in the Anthropic SDK call. No Anthropic API call at this step — pure I/O.*

STEP 2 — EXTRACT

TECHNICAL

The transcript is sent to `claude-sonnet-4-6` via `POST /v1/messages`. The system prompt defines the model role as Senior CSM Analyst and enforces a strict JSON output schema. Insight types are fully enumerated: `BUG_REPORT`, `COMPETITOR_MENTION`, `FEATURE_REQUEST`, `PRICING_FRICTION`, `CHURN_SIGNAL`, `POSITIVE_SIGNAL`, `GENERAL_FEEDBACK`. The model cannot return a type outside this set. The prompt is versioned as `SYSTEM_PROMPT_v1.0.0` to prevent schema drift.

NON-TECHNICAL / EXECUTIVE

Claude reads the entire conversation and pulls out every signal that matters — like having your sharpest analyst on every single call, never tired, never distracted, always writing the same structured report. A CSM catching notes manually gets about 60% of signals. This system catches 100%, every call, in exactly the same format.

API / SDK: *`client.messages.create(model="claude-sonnet-4-6", system=SYSTEM_PROMPT_v1, messages=[{"role": "user", "content": transcript}], max_tokens=2000)`. Model string in the React POC: `claude-sonnet-4-20250514`. This is the primary Anthropic API call — all insight types extracted in a single request.*

⚠️ GAP vs. INVOCA: *Invoca's Signal AI uses proprietary supervised ML models trained on Databricks plus an LLM layer. Signal AI Studio is the no-code productized version for non-technical marketers. Your POC is the engineer's version — full schema control, versioned prompts, enumerated types. You speak the same language as their platform team.*

STEP 3 — SCORE

TECHNICAL

Every `ExtractedInsight` returned by the model

NON-TECHNICAL / EXECUTIVE

Every insight has a score from 0 to 100%

includes `confidence_score` as a required first-class field — a float from 0.0 to 1.0. The calibration table is baked into the system prompt: 0.95+ = explicit evidence, 0.85–0.94 = clear signal, 0.75–0.84 = reasonable inference, below 0.75 = uncertain. The model cannot omit this field — the schema enforces it. Confidence is first-class, not inferred post-hoc from the text.

showing how certain the system is. Above 75% it acts automatically. Below 75% a human reviews first. Like a weather forecast — 95% chance of rain means bring an umbrella, 55% means wait for a human call. The system never silently guesses. It always declares how sure it is.

API / SDK: `confidence_score` is a required field in the `ExtractedInsight` dataclass: `@dataclass class ExtractedInsight: confidence_score: float`. Enforced in the system prompt. No separate API call — returned in the same response as the structured insight. One API call, all scoring included.

⚠️ GAP vs. INVOCA: Invoca's Signal AI shows accuracy scores in Signal AI Studio and uses them as deployment thresholds — a signal can't go live until it clears a minimum accuracy bar. Your scoring runs per-insight at inference time on every call, not as a one-time deployment gate. More granular, continuous, and auditable.

🔁 STEP 4 — GATE

🔧 TECHNICAL

A single deterministic Python conditional: if `insight.confidence_score >= 0.75`: `route(insight)` else: `human_review_queue.append(insight)`. No ML at the gate layer — pure conditional logic. Stateless and fully auditable. The same input always produces the same routing decision. The 0.75 threshold is configurable per `insight_type` for business rules requiring different gates per signal category.

👤 NON-TECHNICAL / EXECUTIVE

Before anything gets sent to anyone, the system asks one question: are we sure enough? This is what separates a system people trust from one they ignore. Uncertainty is surfaced, not hidden. Like a spell-checker that flags uncertain words instead of silently guessing — the gate is the moment the system admits what it does not know.

API / SDK: Pure Python — no additional Anthropic API call. `def gate(insight): return route(insight) if insight.confidence_score >= 0.75 else human_queue.append(insight)`. Deterministic, version-controlled, fully unit-testable. The React POC renders the 75% threshold line explicitly in the Confidence Visualization panel.

⚠️ GAP vs. INVOCA: Invoca's equivalent is the Signal AI Studio deployment threshold — a signal model cannot go live without clearing a minimum accuracy score. Key difference: your gate fires per-insight on every live call at runtime. Invoca's threshold fires once at model deployment time. Yours is runtime quality control. Theirs is launch quality control.

🚦 STEP 5 — ROUTE

🔧 TECHNICAL

A Python dictionary — `ROUTING_RULES` — maps each `InsightType` enum to a tuple of (destination_team, sla_string). A `RoutedAlert` dataclass is created per insight with full

👤 NON-TECHNICAL / EXECUTIVE

Once the system is confident in an insight, it sends it to exactly the right team with a deadline attached. Engineering gets bugs. Sales gets competitor flags. CS Leadership gets churn

metadata. Four destinations: Engineering, CS Leadership, Sales Leadership, Product Management. SLA assigned deterministically from urgency: CRITICAL → 4 hours, HIGH → 48 hours, MEDIUM/LOW → 1 week. The dictionary is version-controlled alongside the schema.	risks. Product gets feature requests. Like a smart receptionist who reads every message, knows exactly who handles it, and puts it in their inbox with a due date — before anyone asks.
API / SDK: Python <code>ROUTING_RULES</code> dict lookup — no additional Anthropic API call. Output is the <code>RoutedAlert</code> dataclass serialized to JSON. Webhook-ready for POST to Salesforce task creation, Slack message, or Jira ticket. The React POC renders this as the Stakeholder Routing Map kanban panel.	
 GAP vs. INVOCA: Invoca's Invoca Exchange routes signal data to 200+ external platforms — Google Ads, Salesforce, Meta, Adobe. Architectural concept is identical: signal fires downstream action. Your POC routes to 4 internal teams by design. Your Phase 2 roadmap (Salesforce + Slack write-back) closes this integration gap directly.	



STEP 6 — ALERT

 TECHNICAL RoutedAlert objects serialized to JSON. Required fields: alert_id (UUID), destination team, urgency level, SLA string, structured summary, and verbatim_quote. verbatim_quote is a required schema field — the model cannot omit the exact customer words that triggered the insight. This is architectural: hallucination cannot fabricate an evidence trail because the quote field is schema-enforced and non-nullable. Output renders to the React dashboard and is push-ready via webhook.	 NON-TECHNICAL / EXECUTIVE The right team gets a complete brief — not a sticky note. What the customer said, exactly why it matters, and when they need to respond. The customer's own words are always attached. When you take a churn risk to your VP, you are not paraphrasing an AI summary — you are quoting the customer directly. That is the difference between a system people trust and one they question.
API / SDK: JSON serialization of the <code>RoutedAlert</code> dataclass. <code>verbatim_quote</code> is required in the Anthropic system prompt — the model is explicitly instructed it cannot omit this field. The React POC renders <code>InsightCard</code> components with verbatim blockquotes. The same JSON is webhook-ready for Salesforce opportunity notes or Slack Block Kit messages.	



STEP 7 — CONFIRM

 TECHNICAL The CSM receives a structured confirmation receipt per insight: alert_id, destination team, SLA deadline (computed from urgency level + timestamp), suggested action string, and loop_status: CLOSED. This closes the feedback loop — the CSM has proof the insight was extracted, where it went, who owns it, and what	 NON-TECHNICAL / EXECUTIVE The CSM gets a receipt. Here is what we found. Here is where it went. Here is the deadline. Loop closed. No guessing whether Engineering saw the ticket. No following up to find out if Sales got the competitor flag. Like a read-receipt on a critical email — you know it was delivered, you know who has it, you know the
---	---

happens next. Enables downstream SLA compliance tracking. The React POC shows 6/6 confirmed in the ClosedLoopConfirmations panel.	due date.
API / SDK: Confirmation object rendered client-side in the React dashboard. SLA deadline computed from urgency + alert timestamp — no additional Anthropic API call. Derived state from the RoutedAlert object. Push-ready to Salesforce task due date or Slack reminder in Phase 2.	
⚠️ GAP vs. INVOCA: THIS IS THE PRIMARY GAP. Invoca's closed loop is revenue attribution — campaign click to call to offline conversion to ad platform write-back. Answers: did this campaign generate revenue? Your closed loop answers: did the right person get this insight, do they own it, is there a deadline? Invoca has no confirmation receipt, no ownership assignment, no SLA enforcement, and no loop-closed status in the CS workflow layer. You built that.	

SECTION 3: The Gap — What Your System Adds That Invoca Lacks

Precision matters here. You are not criticizing Invoca. You are identifying a specific accountability gap in their CS workflow layer that your system addresses — and that complements their platform.

 INVOKA'S CLOSED LOOP Revenue Attribution Marketing campaign click → call → offline conversion → ad platform write-back. Pushes revenue data to Google Ads and Salesforce. Answers: did this campaign generate revenue?	 YOUR CLOSED LOOP Workflow Accountability Call insight → named team → SLA deadline → ownership confirmation → loop-closed receipt. Answers: did the right person get this, do they own it, is there a deadline?
---	---

These are two different loops. Invoca closes the marketing attribution loop. Your system closes the CS accountability loop. They are complementary — not competitive.

What Invoca's Platform Does NOT Include

- No structured confirmation receipt to the CSM after insight routing
- No SLA deadline per insight based on urgency level — Smart Alerts notify but do not set deadlines
- No ownership tracking — no confirmation which human owns each insight
- No loop-closed status — no audit trail confirming an insight was received and actioned
- No cross-functional internal routing to CS, Engineering, Sales, and PM as distinct accountability targets
- QA Scorecards score agent behavior for compliance — they do not route extracted insights to stakeholders

Say This in the Room

"Invoca's Signal AI does something I deeply respect — it extracts structured signals from calls at enterprise scale with patented ML models and a no-code studio for non-technical users. My system is the architect's version of that same extraction pipeline. But there is a specific gap Signal AI, Smart Alerts, and Invoca Exchange are not filling: the internal accountability loop. Invoca closes the loop between marketing spend and call outcome. My system closes the loop between a call insight and a stakeholder action. Those are two different loops. Right now, Invoca only has one of them."

Gap Evidence — Feature by Feature

Invoca Feature	What It Does	What It Lacks (Your Gap)
Signal AI	Extracts signals from calls using Databricks-trained ML + LLM. Assigns signal types and accuracy scores.	No internal routing to named CS/Eng/Sales/PM teams. No SLA. No confirmation receipt. Signals route to ad platforms, not CS workflows.
Smart Alerts	Notifies configured recipients when signal conditions are met via webhook or email.	Notification is not confirmation. No acknowledgment layer. No ownership assignment. No deadline enforcement.
Signal AI Studio	No-code interface for marketers to create and deploy custom signals with accuracy thresholds.	Deployment threshold gates model launch — not per-call inference. No runtime accountability loop after the signal fires.
Closed-Loop Attribution	Connects call back to marketing campaign. Pushes revenue data to Google Ads and Salesforce opportunity stages.	Revenue attribution loop only. No internal CS accountability layer. No loop-closed status for CSM workflow.
QA Scorecards	Post-call agent compliance scoring for contact center quality assurance.	Scores agent behavior — does not route extracted insights to cross-functional stakeholders with SLA deadlines.

SECTION 4: Speaking Precisely About the Anthropic API

Exactly how to describe the API usage — confidently, accurately, without overstating or understating.

The Exact API Call

```
# Python SDK
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=2000,
    system=SYSTEM_PROMPT_v1, # versioned constant – not inline string
    messages=[{"role": "user", "content": transcript}]
)

# React POC – direct client-side call (production: proxy for key
# protection + PII redaction)
fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  body: JSON.stringify({ model: "claude-sonnet-4-20250514",
    system: SYSTEM_PROMPT_v1, messages:
    [{role:"user",content:transcript}] })
})
```

Key Technical Points — Say These Confidently

- Model: claude-sonnet-4-6 — Anthropic's latest Sonnet-class model as of Q1 2026. Strong intelligence-to-latency balance for production pipelines. The React POC uses claude-sonnet-4-20250514 — same model family.
- System prompt as a named constant (SYSTEM_PROMPT_v1) — not an inline string. This enables version control, prevents schema drift, and makes the prompt auditable like any other code artifact.
- Schema enforcement in the prompt — the system prompt contains the full JSON schema with enumerated insight types. The model is instructed to return ONLY the JSON object — no preamble, no commentary.
- confidence_score is a required field — declared in the system prompt as non-nullable. The model cannot omit it. First-class confidence scoring, not post-hoc inference.
- verbatim_quote is required — the model must return the exact customer sentence that triggered each insight. Eliminates hallucination risk in the evidence trail. Schema-enforced, not best-effort.
- One API call, all insights — multiple ExtractedInsight objects returned in a single response as a JSON array. Efficient by design.
- Response parsing — data.content[0].text mapped for text blocks. POC uses regex fallback (text.match(/\[([s\S]*)\]/)) for safe JSON extraction. Production uses structured outputs.

Your API Usage vs. Invoca's Signal API

Layer	Your POC (Anthropic SDK)	Invoca Signal API
Endpoint	POST	POST /signals via Invoca platform API

	https://api.anthropic.com/v1/messages	
Model	claude-sonnet-4-6 (Anthropic)	Proprietary ML + LLM on Databricks
Schema	Versioned system prompt, enumerated types, strict JSON	Named signals defined in Signal AI Studio interface
Confidence	confidence_score: float, required field, calibration table in prompt	Accuracy score in Studio UI, per-signal deployment threshold
Output	ExtractedInsight JSON array — all insights in one call	Signal match data — per-transaction signal responses
Routing	ROUTING_RULES dict — 4 teams + deterministic SLA	Invoca Exchange — 200+ platforms, webhook delivery
Loop Close	CSM receipt, loop_status: CLOSED	Revenue attribution push to ad platforms — no CS workflow loop
Production Gap	Add PII redaction layer pre-API call (Phase 2)	Full PCI DSS + HIPAA compliance at ingestion layer

SECTION 5: Full Alignment Map — POC vs. Platform

Use this to show you understand Invoca's platform deeply — not just your own system.

Your Step	Invoca Equivalent	Alignment	Your Interview Edge
1. Ingest	Call Ingestion API + telephony routing	STRONG ALIGN	<i>Acknowledge their PCI/HIPAA compliance layer at ingestion. Name it. Add it to your Phase 2 roadmap statement.</i>
2. Extract	Signal AI — Databricks ML + LLM	STRONG ALIGN	<i>"My POC is the engineer's version of Signal AI Studio." That line works in every technical conversation.</i>
3. Score	Signal accuracy scores in Studio	EXACT MATCH	<i>Use Invoca's own language: 'not a black box.' They say it in their docs. You built the same principle independently.</i>
4. Gate	Studio deployment accuracy threshold	STRONG ALIGN	<i>Your gate fires per-call at runtime. Theirs fires once at model deployment. Yours is more granular.</i>
5. Route	Invoca Exchange — 200+ integrations	PARTIAL ALIGN	<i>Acknowledge the scale gap honestly, then point to Phase 2. That shows maturity, not weakness.</i>
6. Alert	Smart Alerts + AI summaries + transcripts	STRONG ALIGN	<i>verbatim_quote is required in your schema — stronger guarantee than surfacing transcripts alongside alerts.</i>
7. Confirm	None — revenue attribution	YOU GO	<i>Primary differentiator. Say it clearly.</i>

	loop only	FURTHER	Then stop talking.
--	-----------	---------	--------------------

SECTION 6: Closing Lines — Memorize These Three

Three lines. One for each room type. Say them clearly, then stop.

For a Technical Panel or Engineering Audience

"I built the engineer's version of Signal AI — same extraction philosophy, full schema control, versioned prompts via the Anthropic SDK, and a confidence gate that fires on every call at inference time. The piece I added that Signal AI does not have is the internal accountability loop: insight extracted, team assigned, SLA set, loop confirmed closed."

For an Executive, VP, or Product Audience

"Invoca closes the loop between marketing spend and call outcome. My system closes the loop between call insight and stakeholder action. Right now, Invoca only has one of those loops. I built the other one."

For Any Room — The Final Sentence

"The insight was always in the conversation. The problem was not data — it was workflow. I built a system that turns conversations into structured, accountable action." Then stop talking.